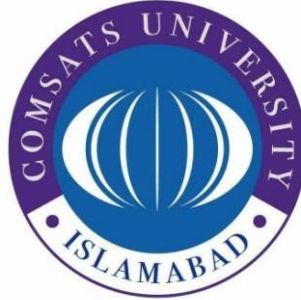


COMSATS University Islamabad

Attock Campus



Department Of Computer Science

Student Details

<i>Registration No.</i>	<i>Name</i>
<i>FA24BSE041</i>	<i>ROMASSA EMAN</i>

LAB ASSIGNMENT#01

CODE:

```

1  def caesar_encrypt(text, shift): 2 usages
2      result = ""
3      # We loop through every single character in your message
4      for char in text:
5          # 1. Handle Uppercase letters (A-Z)
6          if char.isupper():
7              # ord(char) gets the ASCII number (A=65).
8              # We subtract 65 to start at 0, add the shift,
9              # use % 26 to wrap Z back to A, then add 65 back.
10             result += chr((ord(char) + shift - 65) % 26 + 65)
11         # 2. Handle Lowercase letters (a-z)
12         elif char.islower():
13             # Same logic, but using 97 (the ASCII start for 'a')
14             result += chr((ord(char) + shift - 97) % 26 + 97)
15         # 3. Handle spaces, numbers, and symbols (like @ or !)
16         else:
17             # These characters are not shifted
18             result += char
19     return result
20 def caesar_decrypt(ciphertext, shift): 1 usage
21     # Decrypting is simply reversing the shift (moving backwards)
22     return caesar_encrypt(ciphertext, -shift)
23 # --- Execution of the message ---
24 # Your specific input message
25 message = "Lets go to Islamabad for the project of Ideology @2026"
26 shift_value = 4 # This means 'a' becomes 'e', 'b' becomes 'f', etc.

```

```

27     # Run the encryption
28     encrypted = caesar_encrypt(message, shift_value)
29     # Run the decryption to prove it works
30     decrypted = caesar_decrypt(encrypted, shift_value)
31     # Print the results
32     print(f"Original: {message}")
33     print(f"Encrypted: {encrypted}")
34     print(f"Decrypted: {decrypted}")

```

OUTPUT:

```
"C:\Users\HP Folio 9470m\AppData\Local\Programs\Python\Python311\python.e
Original: Lets go to Islamadad for the project of Ideology @2026
Encrypted: Pixw ks xs Mwpeqeheh jsv xli tvsnigx sj Mhispskc @2026
Decrypted: Lets go to Islamadad for the project of Ideology @2026

Process finished with exit code 0
```

Line-by-Line Code Explanation:

1. The Encryption Function

```
def caesar_encrypt(text, shift):
```

- **Purpose:** Takes the raw `text` and a numerical `shift` value to produce a scrambled message.

```
for char in text:
```

- The code iterates through the string one character at a time. This ensures that every letter, space, and symbol is processed.

```
if char.isupper(): / elif char.islower():
```

- The code treats uppercase and lowercase differently to preserve the original casing of your message.

```
result += chr((ord(char) + shift - 65) % 26 + 65)
```

- This is the mathematical "heart" of the cipher. Let's break down the logic:
- `ord(char)`: Converts a letter to its ASCII numeric value (e.g., 'A' is 65).
- `- 65`: Normalizes the value so 'A' becomes 0, 'B' becomes 1, and so on. This makes the math easier.
- `+ shift`: Moves the letter forward by the specified amount.
- `% 26`: The **Modulo** operator. This is crucial—it handles "wrap-around." If you shift 'Z' (25) by 1, it becomes 26. $26 \pmod{26} = 0$, sending you back to 'A'.
- `+ 65`: Shifts the number back into the ASCII range for uppercase letters.

- `chr(...)`: Converts that final number back into a character.
`else: result += char`
- This handles "non-alphabetic" characters (like the `@` or `2026` in your message). It simply appends them to the result without changing them.

2. The Decryption Function

```
def caesar_decrypt(ciphertext, shift):
```

- **Logic:** Decrypting a Caesar cipher is just "encrypting in reverse." By passing `-shift` into the encryption function, you move the letters backward to their original positions.

Security Analysis:

While this code is a great exercise in logic and string manipulation, it is **completely insecure** by modern standards. Here is why:

- **Tiny Key Space:** There are only **25 possible shifts** in the English alphabet (a shift of 26 returns the original text). A computer can "Brute Force" (try every possibility) this in less than a millisecond.
- **Frequency Analysis:** In English, certain letters (like 'E', 'T', and 'A') appear more often than others. Even if the shift is unknown, a hacker can look at which character appears most frequently in the encrypted text and guess the shift.
- **No Integrity Check:** There is no way to verify if the message was tampered with during transmission.
- **Pattern Preservation:** Because the same letter always encrypts to the same ciphertext letter, the "shape" of the words remains visible. Spaces and punctuation are left unencrypted, providing clues to the message structure.